

New Construction on Zk-Creds Credential System

Anonymous Submission

Abstract. Credential revocation remains a major unresolved challenge within cryptographic systems. Existing solutions rely on ever-expanding delta lists, leading to system degradation over time due to data nullification and redundancy. Furthermore, any update to the underlying cryptographic primitive forces the prover to entirely recompute their proof. To address these inefficiencies, we propose an alternative construction on credential systems based on skip lists. Our approach significantly reduces prover computation time, minimizes proof recomputation overhead, and provides an efficient, scalable mechanism for credential revocation. Notably, while this construction is focused on the zk-creds construction, it is fully generalizable and can be readily extended to broader cryptographic primitives and authenticated data structures requiring dynamic updates.

Keywords: ZK-creds · Skip List · Revocation · Credentials · Algorithm

1 Introduction

In credential systems, Zero-knowledge proofs typically means that we are proving membership to a list that is issued by either a trusted issuer or a completely transparent board. In the zk-creds paper [RWGM22], this credential membership system is done via a forest of merkle trees. Although this construction has successfully achieved the goal of real world deployment of zk systems and proving time optimization via the forests, Credential revocation in said system remained unsolved and was left as future work. Which this proposed solution would attempt to increase complexity optimization, proving optimization, and propose a possibly better solution to revocation.

Whether a person wants to prove that they are an employee at a company, proving that they are a citizen of a certain country, or proving that they possess a valid X509 certificate, simply showing the related data leaks a lot of information and subsequently violating privacy. For example LinkedIn’s verification protocol that checks your birth date, country of issuance, etc. With zero-knowledge credential systems, this would allow the same functionality with no loss of integrity without sacrificing privacy or individual “unit” data.

One of the most prominent related work in this domain is a construction called the Cartesian Merkle Tree proposed in [CKR25]. It combines the properties of a Binary Search Tree, a Heap, and a Merkle tree, and most importantly supports insertion and deletion. However, the sorted nature and its complexity is making it quite hard to realistically use in the real world. Furthermore, sorted variants of merkle trees are known to have less security than normal ones.

1.1 Current Problems

Several problems to be addressed and what other works have been done to address them.

Revocation. Efficient revocation remains fundamentally unfeasible in standard Merkle tree constructions due to their structural rigidity. Existing workarounds typically rely on external delta lists, blacklists, or direct node nullification within the tree. However, these

approaches introduce severe performance trade-offs. In traditional Merkle tree architectures, nullifying a node is highly inefficient; because the data structure is static, every individual revocation forces the system to recompute the entire authentication path from the affected leaf up to the root. Consequently, this creates unsustainable computational overhead and proof-maintenance bottlenecks as the dataset scales.

Furthermore, this problem is compounded by the specific architectural choices of the zk-creds protocol. The zk-creds “merkle forest” construction periodically merges separate trees T_i (for $i \in \{1, \dots, n\}$) into a unified tree T_{n+1} . Because of this continuous consolidation, true structural deletion is functionally impossible, leaving node nullification as the only viable mechanism. Under frequent revocations, this causes the protocol’s original optimization benefits to decay. The tree suffers from severe structural degradation over time, accumulating a geometric volume of redundant, nullified nodes that permanently bloat the data structure and degrade overall system efficiency.

To circumvent the inefficiencies of in-place node nullification, one might consider adapting Certificate Revocation Lists (CRLs)—akin to the RFC 5280 standard commonly utilized in traditional X.509 public key infrastructures. While a CRL-based mechanism avoids direct modifications to the underlying membership list, it introduces severe architectural and privacy-preserving conflicts within anonymous credential systems.

First, standard CRL frameworks fundamentally depend on the existence of a unique, persistent identifier to flag revoked certificates. For a cryptographic protocol like zk-creds, enforcing such an identifier requires embedding a static, pseudo-anonymous tag (such as a credential hash or a committed attribute) directly within the credential. This creates a precarious privacy trade-off, as revocation relies on either the user voluntarily exposing this identifier or a central auditor possessing the ability to de-anonymize malicious or misused credentials.

Second, importing CRL mechanics introduces significant practical and systemic vulnerabilities. Maintaining a synchronized and secure revocation list demands strict update granularity and cross-network consistency, ensuring all distributed verifiers maintain an identical state. Furthermore, delegating the management of this list to a designated issuer or authority re-introduces centralized trust assumptions—specifically regarding the honesty and key security of the managing party—which actively undermines the decentralized, zero-knowledge ethos of the zk-creds protocol.

Proof Recomputation Constraints. In cryptographic frameworks utilizing a single, monolithic data primitive, any state update inherently necessitates that provers recompute their membership proofs. While the *zk-creds* Merkle forest construction mitigates the frequency of these updates by decoupling independent trees, the proof recomputation overhead remains a substantial bottleneck for active provers. We argue that this performance boundary can be optimized further.

2 Preliminary

2.1 Skip List

A skip list, originally introduced by Pugh[Pug90], is a probabilistic, layered data structure that allows for efficient search, insertion, and deletion operations. Structurally, it serves as an alternative to balanced binary search trees, achieving comparable logarithmic time complexities without requiring complex rebalancing algorithms.

Formally, a skip list $\mathcal{S}$ consists of a series of ordered linked lists $\{L_0, L_1, \dots, L_h\}$ arranged in a hierarchy of levels, where h denotes the maximum height of the list. The structural properties of $\mathcal{S}$ are governed by the following invariants:

1. **Base Layer (L_0):** The bottom-most layer, L_0 , is a standard sorted linked list

92 containing all elements in the data set, bounded by special sentinel nodes representing
 93 $-\infty$ and $+\infty$.

94 **2. Subsampling Hierarchy (L_i):** For each successive level $i > 0$, the list L_i contains
 95 a sparse subset of the elements found in L_{i-1} , acting as an express lane for traversal.
 96 If an element x belongs to L_i , it maintains a downward pointer to its corresponding
 97 representation in L_{i-1} .

98 **3. Probabilistic Promotion:** The height of a newly inserted element is determined
 99 probabilistically via independent coin flips. An element is promoted from level i
 100 to level $i + 1$ with a fixed probability p (commonly chosen as $p = 1/2$ or $1/4$),
 101 independent of the total number of elements n .

102 To search for a target key k , the traversal originates at the top-most sentinel node of L_h
 103 and proceeds horizontally along the current layer until the next node's key exceeds k . At
 104 that boundary, the search drops down vertically to the next layer (L_{i-1}) via the downward
 105 pointer. This process repeats until the element is located or the search terminates at the
 106 base layer L_0 .

Because the number of elements decreases geometrically at each higher level, the expected height of the skip list is bounded by $\mathcal{O}(\log n)$. Consequently, the expected time complexity for search, insertion, and deletion operations is proven to be:

$$\mathbb{E}[\text{Time}] = \mathcal{O}(\log n)$$

107 Although, the worst-case space complexity of the structure scales linearly as $\mathcal{O}(n)$ it has
 108 been proven that this is encountered on negligible chance. Analysis and proofs of these
 109 complexity bounds can be found in the foundational work by Pugh [Pug90].

110 2.2 Zero-Knowledge Proof Systems

111 A Zero-Knowledge (ZK) proof system is a cryptographic primitive that enables a party,
 112 designated as the *prover* ($\mathcal{P}$), to convince another party, the *verifier* ($\mathcal{V}$), of the validity of
 113 a mathematical statement without revealing any information beyond the statement's truth
 114 itself. Formally, let $\mathcal{R}$ be a polynomial-time decidable relation. We say that x is a valid
 115 statement if there exists a secret piece of data, called a *witness* w , such that $(x, w) \in \mathcal{R}$.
 116 The corresponding language is defined as $\mathcal{L} = \{x \mid \exists w \text{ s.t. } (x, w) \in \mathcal{R}\}$.

117 An interactive or non-interactive proof system for a language $\mathcal{L}$ must satisfy three
 118 fundamental security properties:

1. **Completeness:** If the statement is true and both the prover and verifier honestly follow the protocol, the verifier will accept the proof with overwhelming probability. Formally, for every $(x, w) \in \mathcal{R}$:

$$\Pr[\mathcal{V}(\mathcal{P}(x, w), x) = 1] = 1$$

2. **Soundness:** If the statement is false, no malicious prover $\mathcal{P}^*$ can convince an honest verifier that the statement is true, except with a negligibly small probability (the soundness error ϵ). Formally, for any $x \notin \mathcal{L}$ and any adversarial $\mathcal{P}^*$:

$$\Pr[\mathcal{V}(\mathcal{P}^*(x), x) = 1] \leq \epsilon$$

3. **Zero-Knowledge:** The proof leaks absolutely no information about the secret witness w . Formally, the system is zero-knowledge if, for every PPT verifier $\mathcal{V}^*$, there exists a simulator $\mathcal{S}$ that can produce a simulated transcript on input x alone (without access to w) that is computationally indistinguishable from a real interaction between $\mathcal{P}$ and $\mathcal{V}^*$:

$$\{\text{View}_{\mathcal{V}^*}[\mathcal{P}(x, w) \leftrightarrow \mathcal{V}^*(x)]\} \approx_c \{\mathcal{S}(x)\}$$

3 Definitions

3.1 Authenticated Data Structure

An Authenticated Data Structure (ADS) has a formal definition such as given by Tamasia[Tam03]. However, we are going to use the more simplified and practical definition of what an ADS is in systems.

An ADS is a data structure in which each node stores, in addition to its data, a *digest* $h_v = H(d_v \parallel h_{c_1} \parallel \dots \parallel h_{c_k})$, where $H : \{0, 1\}^* \rightarrow \{0, 1\}^\lambda$ is a collision-resistant hash function, d_v is the data held at node v , and $h_{c_1}, \dots, h_{c_k}$ are the digests of its children. The digest of the root, h_{root} , serves as a compact, tamper-evident commitment to the entire structure.

A *membership proof* (or *authentication path*) for a node v consists of the sibling digests along the path from v to the root, allowing any verifier who holds h_{root} to recompute and confirm the root digest in $\mathcal{O}(\log n)$ hash operations.

3.2 Interval Based Skip List (IBSL)

An Interval Based Skip List or IBSL for short, is a new version of skip list. It serves at worst the efficiency of a normal skip list and on average should be faster than a normal skip list. It is based on the Rank Based Skip List proposed in [SDR16], however, it is interval based.

An IBSL in its core is similar to a normal skip list in that it is a series of ordered linked lists $\{L_0, L_1, \dots, L_h\}$ arranged in a hierarchy levels. However, in each level L_i for $i > 1$, the linked lists may be detached in parts and have shortcuts to nodes in linked lists of L_j where $j < i$.

Formally, an IBSL is a modified skip list such that it has the following properties:

1. **Authenticated:** An IBSL is an authenticated skip list such that each interval node have an additional data of a *digest* $h_v = H(RP \parallel RD)$ where RP is the digest inside the right pointer node and RD is the digest inside the down pointer node.
2. **Shortcut Paths:** An IBSL is a skip list that has the concept of shortcut nodes, which are right pointers that does not go to the next node in the i^{th} level but goes down. (cleared up in Appendix)
3. **Interval Based:** An IBSL is a skip list that is similar to a Rank Based Skip List, however, instead of Ranks it stores Intervals.
4. **Individual Root:** An IBSL would always have a level h that is the highest level which only has 1 node which is the root node.

Furthermore, an IBSL would have the following algorithms:

Setup($1^\lambda, \{com_i\}_{i=1}^n$) $\rightarrow \sigma$ Compute node intervals and hashes for each node $N_i \in L_i$. Build the skip list bottom-up. Outputs $\sigma \leftarrow \mathcal{S}_{\text{root}}$ Then optimize skip list.

Prove($\mathcal{S}, k$) $\rightarrow (r, \pi)$ Let v be the leaf for key k , set $r \leftarrow d_k$ and $\pi \leftarrow \langle \rangle$. While $v \neq \text{root}$: append sibling hash h_s to π , set $v \leftarrow \text{parent}(v)$. Output (r, π) .

Verify(σ, k, r, π) $\rightarrow \{0, 1\}$ Set $h \leftarrow H(r)$. For each $h_s \in \pi$: set $h \leftarrow H(h \parallel h_s)$. Output 1 if $h = \sigma$, else output 0.

Search($\mathcal{S}, k$) $\rightarrow \{0, 1\}$ Let $v \leftarrow \mathcal{S}_{\text{root}}$. While $k \notin v_{\text{interval}}$: $v \leftarrow \text{right}(v)$, otherwise $v \leftarrow \text{down}(v)$. If $v \in L_1$: $v \leftarrow \text{right}(v)$ while $\text{right}(v) < k$. Output 1 if $v = k$, else 0.

Insert($\mathcal{S}, k$) $\rightarrow (r, \pi)$ Let $v \leftarrow \mathcal{S}_{\text{root}}$. While $k \notin v_{\text{interval}}$: $v \leftarrow \text{right}(v)$, otherwise $v \leftarrow \text{down}(v)$. If $v \in L_1$: $v \leftarrow \text{right}(v)$ while $\text{right}(v) \leq k$. Then insert k , recompute hashes, outputs $(\mathcal{S}_{\text{newroot}}, \pi)$, and optimize skip list.

164 $\text{Delete}(\mathcal{S}, k) \rightarrow (r, \pi)$ Invoke $\text{Search}(\mathcal{D}, k)$ and remove v . Output (r, π) if Search
 165 outputs 1, else output $(\text{Null}, \text{Null})$.

166 Refer to appendix A for further detail.

167 **Definition 1** (IBSL Interval Correctness). For an IBSL $\mathcal{S}$ and $j < i$, Interval Correctness
 168 can be defined as

$$169 \forall v \in \mathcal{S} \in L_i : v.\text{interval} = \begin{cases} [\min(\text{down}(v).\text{interval}), \max(\text{right}(v).\text{interval})] & \text{if } \text{right}(v) \in L_j \\ [\min(\text{down}(v).\text{interval}), \min(\text{right}(v).\text{interval})] & \text{if } \text{right}(v) \in L_i \end{cases}$$

170 4 Construction

171 We propose several constructions, each of which may be adopted independently or instan-
 172 tiated as successive stages of a unified scheme. Formally, let $\Pi_1, \Pi_2, \dots, \Pi_k$ denote the
 173 proposed constructions in order. Each Π_i is a stand-alone cryptographic construction that
 174 is secure in isolation; moreover, each Π_i is compatible as a strict refinement of Π_{i-1} , in
 175 the sense that any instantiation of Π_i subsumes the functionality and security guarantees
 176 of all preceding constructions $\Pi_1, \dots, \Pi_{i-1}$.

177 4.1 Monolithic Skip List

178 In the monolithic construction, credential membership is maintained by a single IBSL $\mathcal{S}$
 179 over the set of issued commitments $\{com_1, com_2, \dots, com_n\}$. The root digest $\sigma \leftarrow \mathcal{S}_{\text{root}}$ is
 180 published and serves as the public commitment to the current membership state.

181 **Issuance and Revocation.** One of the problems in currently popular vector commitments,
 182 e.g. Merkle Trees, are that they tend to have trouble in revocation or deletion. The IBSL
 183 construction would still preserve the efficiency of these popularly used vector commitments
 184 but with an additional improvement of revocability.

185 When a new credential com_{new} is issued, the issuer invokes $\text{Insert}(\mathcal{S}, com_{new})$, which
 186 updates $\mathcal{S}$ and produces a new root digest σ' and the proof of validation π . Revocation is
 187 handled symmetrically via $\text{Delete}(\mathcal{S}, com_i)$ except that $\text{Verify}(\sigma, com_i, r, \pi) = 0$.

188 **Membership Proof.** To demonstrate that a credential com_i belongs to the current
 189 membership list, the prover invokes $\text{Prove}(\mathcal{S}, com_i) \rightarrow (r, \pi)$ to obtain an authentication
 190 path π through $\mathcal{S}$. The prover then constructs a zero-knowledge proof Π asserting knowledge
 191 of a valid π with respect to $(\mathcal{S}, \sigma)$, without revealing com_i itself. The verifier accepts if
 192 and only if $\text{Verify}_{\text{ZK}}(\sigma, \Pi) = 1$.

193 **Limitations.** The monolithic construction suffers from two fundamental inefficiencies.
 194 First, every insertion or deletion changes the root digest σ , invalidating all existing
 195 membership proofs simultaneously. Any member wishing to show a valid proof must
 196 therefore re-invoke Prove against the updated $\mathcal{S}$. With n active members and u updates,
 197 the total recomputation cost scales as $\mathcal{O}(u \cdot n \cdot \log n)$, which is unsustainable at scale.
 198 Second, the height of $\mathcal{S}$ grows as $\mathcal{O}(\log n)$ (see Appendix A), meaning that individual
 199 proof generation and verification times increase as the membership list grows.

200 4.2 Skip List Forest

201 Rather than maintaining a single monolithic IBSL $\mathcal{S}$, we propose a construction in which
 202 credential membership is managed by a forest of IBSLs $\mathcal{F} = \{\mathcal{S}_1, \mathcal{S}_2, \dots, \mathcal{S}_k\}$, extending
 203 the single IBSL construction of Section 4.1. Each $\mathcal{S}_i \in \mathcal{F}$ maintains its own root digest
 204 $\sigma_i \leftarrow \mathcal{S}_i.\text{root}$, and the set of all root digests is defined as $\mathcal{F}.\text{roots} = \{\sigma_1, \sigma_2, \dots, \sigma_k\}$.

Membership Proof. To demonstrate membership, the prover must show that their credential com_i belongs to some $\mathcal{S}_i \in \mathcal{F}$. Concretely, the prover computes an authentication path π through $\mathcal{S}_i$ and constructs a zero-knowledge proof Π asserting knowledge of a valid π with respect to some $\sigma_i \in \mathcal{F}.\text{roots}$, without revealing which tree $\mathcal{S}_i$ the credential belongs to. The verifier accepts if and only if $\text{Verify}(\mathcal{S}_i, \Pi) = 1$.

Optimization. By analogy with the Merkle forest construction of [RWGM22], the forest instantiation yields two key improvements over the monolithic construction. First, an update to any single $\mathcal{S}_i$ invalidates only the membership proofs of credentials within $\mathcal{S}_i$, rather than the proofs of all n members across the entire list. Second, individual proof generation time is reduced, as each $\mathcal{S}_i \in \mathcal{F}$ maintains approximately $\frac{n}{|\mathcal{F}|}$ credentials, yielding an expected height of

$$\mathcal{S}_i.\text{height} \approx \log\left(\frac{n}{|\mathcal{F}|}\right) = \log n - \log |\mathcal{F}| \leq \log n,$$

(appendix ??) which is strictly smaller than the $\mathcal{O}(\log n)$ height of a monolithic IBSL over all n credentials. Thus, both proof generation and verification benefit from a logarithmic reduction proportional to the size of the forest.

4.3 Credential Revocation

A major problem we aim to solve is the problem of credential revocation where current constructions simply null the “node” or simply add the credential to a blacklist. This however as aforementioned presents several severe issues as the amount of credentials increase. First and biggest issue is data degradation, since the structure of the data being kept isn’t updated in accordance to the revocation, as the amount of revocations increase the amount of redundant or “useless” data that exists inside the structure is also increasing. If we have a structure that is housing 100 credentials then even just 50 revocations would make half of the structure useless. With null nodes, the structure would then have a lot of holes which would reduce the efficiency of proving and verification. If we use a blacklist, then not only would we need two proofs such that you have to prove you are in the membership structure and you are not in the blacklist, but the main problem with this approach is that the blacklist would also be ever increasing, wasting storage and increasing both proof generation for both statements.

the IBSL solves this problem because of the underlying construction $\mathcal{O}(\log n)$ complexity for node deletions and thus credentials would be able to simply be removed. Not nulled, not an ever expanding blacklist, just gone. Preserving the integrity of the structure and efficiency as well as reducing storage.

However, note that since we do things in batches to reduce linkability via side channel attacks, the revocation would also be done in batches. The problem with this is that revocation needs to take place immediately, so we propose a solution that uses a “refreshing delta list”.

Refreshing Delta List. How this works is for epoch e and delta list Δ , every credential being revoked would simply be added to Δ and not the structure itself. Then, once e is over and now moves to the next epoch e' , the batch revocation operation is being computed by concretely revoking all credentials in Δ to the structure. Then Δ is reset and is again a clear empty list for e' .

Vector commitments such as Verkle Trees would fit this incredibly well because of their smaller witness sizes and flatter tree structures compared to a normal merkle tree.

4.4 Zero-Knowledge Proof Candidate

In regards to Zero-knowledge proofs, we want to invoke a post-quantum zero-knowledge proof system instead of the classical Groth16 in the zk-creds construction. The prover must produce a zero-knowledge proof Π asserting membership in the construction without revealing. Classical zk-proofs such as Groth16 (used in the original zk-creds construction [RWGM22]) rely on the hardness of the discrete logarithm problem, which is vulnerable to quantum adversaries.

Post-Quantum Candidate: zkSTARKs. A promising candidate for a post-quantum zero-knowledge proof system is the *zkSTARK* (Zero-Knowledge Scalable Transparent Argument of Knowledge), introduced by Ben-Sasson et al. [BSBHR18]. Unlike SNARKs, zkSTARKs rely solely on collision-resistant hash functions and information-theoretic arguments, making them conjectured to be secure against quantum adversaries. Furthermore, zkSTARKs are *transparent* in the sense that they require no trusted setup, eliminating the toxic waste problem inherent in pairing-based SNARKs such as Groth16.

Concrete Properties. For the purposes of this construction, we require the zero-knowledge proof system ZK to satisfy the following properties:

1. **Post-Quantum Security.** The security of ZK must not rely on the hardness of factoring, discrete logarithm, or any problem known to be efficiently solvable by a quantum adversary.
2. **Succinctness.** Proof size and verification time must be sublinear in the size of the witness, i.e., the authentication path π .
3. **Transparency.** ZK must not require a trusted setup, as any compromise of setup parameters would undermine the security of the entire credential system.
4. **Zero-Knowledge.** Π must reveal nothing about the credential com_i , the authentication path π , or the specific tree $\mathcal{S}_i \in \mathcal{F}$ to which com_i belongs.

zkSTARKs satisfy all four properties above, making them a natural fit for this construction. We note however that zkSTARK proof sizes are currently larger than those of pairing-based SNARKs.

4.5 Linked Zero-Knowledge Proofs

It is well known that zero-knowledge proof systems require a circuit to encode the statement being proven. This circuit is static: it has no native support for dynamic or data-dependent statements, where the number of computational steps varies depending on the input. As a result, proving such a statement with a single monolithic circuit requires the circuit to be sized for the worst case, inflating both proof size and proving time regardless of the actual size of the instance being proven.

Folding, a technique underlying Incrementally Verifiable Computation (IVC) schemes such as Nova [KST21] and LatticeFold [BC25], addresses this by combining k individual statements into a single folded statement, rather than requiring all k to be proven within one static circuit. We propose applying this technique to the IBSL as follows: for a search path $\mathcal{P} = (v_1, v_2, \dots, v_n)$ through $\mathcal{S}$, each transition from v_i to v_{i+1} is treated as an individual statement ϕ_i , asserting knowledge of a witness w_i that justifies the corresponding step (e.g. a valid right-move, down-move, or shortcut traversal) without being rejected. The resulting sequence of statements $\{\phi_1, \phi_2, \dots, \phi_n\}$ is then folded into a single statement Φ , which is proven using a zero-knowledge proof system such as a zkSTARK, rather than requiring a single circuit sized to the worst-case path length.

LatticeFold For Folding Commitments. Due to the

References

- [BC25] Dan Boneh and Binyi Chen. LatticeFold+: Faster, simpler, shorter lattice-based folding for succinct proof systems. Cryptology ePrint Archive, Paper 2025/247, 2025.
- [BSBHR18] Eli Ben-Sasson, Iddo Bentov, Yinon Horesh, and Michael Riabzev. Scalable, transparent, and post-quantum secure computational integrity. In *IACR Cryptology ePrint Archive*, 2018.
- [CKR25] Artem Chystiakov, Oleh Komendant, and Kyrylo Riabov. Cartesian merkle tree, 2025.
- [KST21] Abhiram Kothapalli, Srinath Setty, and Ioanna Tzialla. Nova: Recursive zero-knowledge arguments from folding schemes. Cryptology ePrint Archive, Paper 2021/370, 2021.
- [Pug90] William Pugh. Skip lists: a probabilistic alternative to balanced trees. *Commun. ACM*, 33(6):668–676, June 1990.
- [RWGM22] Michael Rosenberg, Jacob White, Christina Garman, and Ian Miers. zk-creds: Flexible anonymous credentials from zkSNARKs and existing identity infrastructure. Cryptology ePrint Archive, Paper 2022/878, 2022.
- [SDR16] Binanda Sengupta, Akanksha Dixit, and Sushmita Ruj. Secure cloud storage with data dynamics using secure network coding, 2016.
- [Tam03] Roberto Tamassia. Authenticated data structures. In *Algorithms – ESA 2003*, Lecture Notes in Computer Science. Springer, 2003.

A Interval Based Skip List — Detailed Construction

Before detailing the construction, we clarify several terms used throughout this appendix. A node is said to be *promoted* or *extended* into level L_i if it gains a representation at L_i in addition to its existing representation at L_{i-1} , via the probabilistic promotion step. A *shortcut path* refers to a *right* pointer that, due to the absence of a node at the current level, leads to a node at a lower level rather than the next node at the same level; such a pointer is produced when a node becomes a *redundant node* (defined below). The notation $n.\text{interval}$ denotes the range of credential values reachable beneath n , used to determine which portion of the keyspace n is responsible for. We use **Search** to refer to the standard skip list traversal procedure: beginning at the top-left node, move right while the next node’s key does not exceed the target, otherwise drop down a level, repeating until the target is located at L_1 . A *batch operation* refers to a collection of inserts and/or deletes applied together before any rebalancing is performed, as opposed to rebalancing after every individual operation. Finally, $b \in \{0, 1\}$ in the Insert procedure below denotes the outcome of a single promotion coin flip with success probability p , repeated until failure to determine an inserted node’s final height, rather than a single fixed bit.

Expanding the IBSL definition from section 3.2, a node v in an IBSL would have the following:

$\text{right}(v) \leftarrow$ the node the right pointer of v is pointing to.

$\text{left}(v) \leftarrow$ the node before v that is pointing to v .

336 $down(v) \leftarrow$ the node the down pointer of v is pointing to, typically straight below and
 337 if $v \in L_i$ then $down(v)$ supposed to point to a node in L_{i-1} .

338 A *redundant node* v is a node where $right(v) = 0$ or $NULL$, and is thus treated as a
 339 shortcut path for $z \leftarrow left(v)$ such that $right(z) = down(v)$.

340 $count(v) \leftarrow$ is the amount of immediate child nodes a node can have.

341 **Definition 2** (IBSL Search Correctness). For an IBSL $\mathcal{S}$,

$$342 \quad \forall k \in \{0, 1\}^\lambda : \text{Search}(\mathcal{S}, k) = \begin{cases} 1 & \text{if } k \in \mathcal{S} \\ 0 & \text{if } k \notin \mathcal{S} \end{cases}$$

343 A.1 Interval In Nodes

344 we denote $v.\text{interval}$ as the interval data inside each node $v \in \mathcal{S}$. A node's interval is designed
 345 to be the identifier of which data is contained/reachable by a certain node v . If $v' \leftarrow right(v)$
 346 is a shortcut-path then this means v' is contained in v and thus is reachable through v ,
 347 because of this $v.\text{interval}$ should encapsulate v' as well. Furthermore, notice that all nodes
 348 reachable via v' is also reachable via v , thus $\max(v.\text{interval}) = \max(v'.\text{interval})$. However,
 349 if $r \leftarrow right(v)$ is another node on the same level, $\max(v.\text{interval}) = \min(r.\text{interval})$.

350 A.2 Setup

351 The setup phase $\text{Setup}(1^\lambda, \{com_i\}_{i=1}^n)$ first constructs L_1 by connecting all $\{com_i\}_{i=1}^n$
 352 into a linked list. Then for each $\{L_i\}_{i=2}^h$, sweep through all nodes in L_{i-1} ; each node is
 353 extended by one node into L_i with probability p . All nodes $n_i \in L_i$ lose $right(n_i)$ and
 354 $left(n_i)$ if n_i is extended up to L_{i+1} , except if $n_i \in L_1$. All nodes n with $right(n) = \text{null}$
 355 become a “redundant node” and we set $right(n) = down(n)$. Intervals are then calculated
 356 as $\min(down(n).\text{interval})$ and $\max(right(n).\text{interval})$ if $right(n)$ is a shortcut path; otherwise,
 357 if $right(n), n \in L_i$, the interval is $\min(right(n).\text{interval})$.

358 Notice that there exists a special case, consider 3 consecutive nodes $\{n_1, n_2, n_3\}$ at L_i .
 359 If n_1 and n_3 promote, they both will lose their predecessor pointer and right pointer. Thus
 360 n_2 will be left stranded. To handle this, we could simply merge n_2 into the path of n_1
 361 such that $right(n_2) = down(n_2)$ then $down(n_2) = down(n_1)$ then $down(n_1) = n_2$.

362 A.3 Insert

363 $\text{Insert}(D, k)$ is slightly different and required some changes because of the nature of the
 364 IBSL in comparison to a normal skip list. When k is being inserted, we first do a normal
 365 search to find the closest leaf node v such that $v \leq k$. Then k is inserted by making
 366 $right(k) = right(v)$ and $right(v) = k$. then pick randomly height $b \in \{0, 1\}$ for k and if
 367 $b = 1$ then k is promoted up a level, repeat the step until we get $b = 0$. Once the height is
 368 finalized, then the following cases for the extension of node k at L_b , k' is to be handled in
 369 the following:

- 370 1. If the previous node z of the current k' at the L_b has a connection, then $right(k') =$
 371 $right(z)$ and $right(z) = k'$. (Figure 1)

372 **Lemma 1.** Let $\mathcal{S}$ be an Interval Based Skip List (IBSL) over n elements. The
 373 expected complexity for a case 1 insertion is $\mathcal{O}(1)$

374 *Proof.* Since case 1 behaves exactly like a normal linked list by simply inserting
 375 a node between 2 nodes, the only operation is done is $right(k) = right(z)$ and
 376 $right(z) = k$ where k is the new node and z is the previous node in L_i . Thus the
 377 complexity is a constant, $\mathcal{O}(1)$. $\square$

- 378 2. If z has no further connection at L_b , i.e. $right(z) \in \{n : n \in L_i, i < b\}$, k' would be
 379 dismissed and $right(z) = k$. (Figure 2)

380 **Lemma 2.** *Let $\mathcal{S}$ be an Interval Based Skip List (IBSL) over n elements. The*
 381 *expected complexity for a case 2 insertion is $\mathcal{O}(1)$*

382 *Proof.* Case 2 does a single operation $right(z) = down(k')$ where k is the new node,
 383 k' is the node extension after promotion of k at L_i , and z is the previous node at
 384 L_i . $\square$

- 385 3. If there are no previous node, i.e. the immediate left of k' is a path and not a
 386 node, $right(k') = down(k)$, $down(k') = down(z')$, $down(z') = k'$. where z' is the
 387 “immediate left node” in L_{b+1} . (Figure 3)

388 **Lemma 3.** *Let $\mathcal{S}$ be an Interval Based Skip List (IBSL) over n elements. The*
 389 *expected complexity for a case 3 insertion is $\mathcal{O}(1)$*

390 *Proof.* Case 3 absorbs the new node k' strictly with 3 operations $right(k') =$
 391 $down(k')$, $down(k') = down(z)$, $down(z) = k'$. Thus is a constant $\mathcal{O}(1)$. $\square$

- 392 4. If k' is cutting through a path below, then also add k' at that level. (Figure 4)

393 **Lemma 4.** *Let $\mathcal{S}$ be an Interval Based Skip List (IBSL) over n elements. The*
 394 *expected complexity for a case 4 insertion is $\mathcal{O}(1)$*

395 *Proof.* Case 4 is identical to inserting a node to a linked list, but to 2 different linked
 396 list for the same new node. This does 4 operations $right(k'') = right(z)$, $right(z) =$
 397 k'' , $down(k'') = down(k')$, $down(k') = k''$. Thus a constant $\mathcal{O}(1)$. $\square$

- 398 5. If k is inserted into a node v that is already at $count(v) = k_{\max}$, then v would
 399 overflow. To handle this, v is split into two nodes v_1, v_2 , dividing v 's children roughly
 400 in half such that $count(v_1), count(v_2) \approx k_{\max}/2$, with v_1 keeping v 's original position
 401 and $right(v_1) = v_2$. One thing to note is that for this case, this splitting may be
 402 cascade up to L_h if all nodes in the affected path are already at $k_{\max}$. (Figure 5)

403 **Lemma 5.** *Let $\mathcal{S}$ be an Interval Based Skip List (IBSL) over n elements. The*
 404 *expected complexity for a case 5 insertion is $\mathcal{O}(\log n)$*

405 *Proof.* Let v be the node that is getting split at L_i , v' be the parent of v at L_{i+1} i.e.
 406 $parent(v)$. After v is split into v_1 and v_2 , this means that $count(v')$ has increased by
 407 one. At worst case, if v' is also at $k_{\max}$ then v' would need to be split as well. It is
 408 easy to see how this would continue to propagate up to $\mathcal{S}.height$, we can consider the
 409 split operation as $\mathcal{O}(1)$, and thus the complexity of case 5 is bounded by $\mathcal{S}.height$
 410 which is $\mathcal{O}(\log n)$. Therefore, case 5 would have $\mathcal{O}(\log n)$ complexity. $\square$

411 A.4 Delete

412 Delete(D, k) has a few concerns, realize that at worst case after an arbitrary x amount of
 413 deletion the state of the IBSL may be highly unoptimize. Consider a state that for each
 414 node $y \in L_i$, $count(y)$ is only a single leaf. Thus any operations would be the same as on
 415 L_1 and thus $\mathcal{O}(n)$. Thus, although there are no specific algorithm for deletion, rebalancing
 416 of the IBSL needs to be done after deletions.

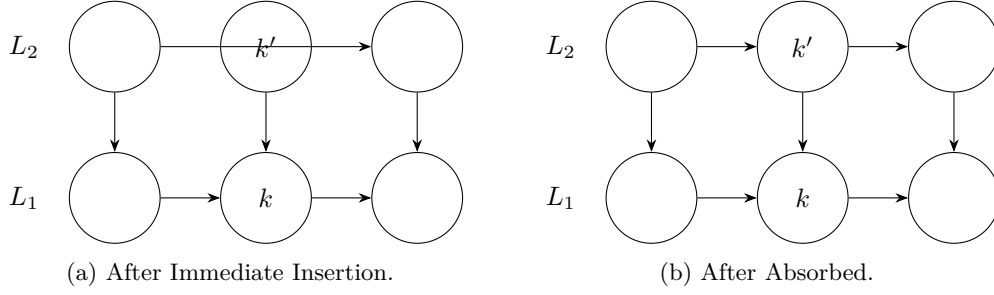


Figure 1: Insertion case 1 for the IBSL.

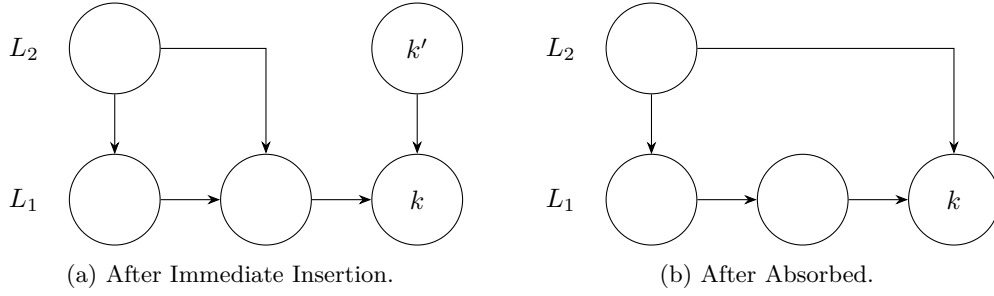


Figure 2: Insertion case 2 for the IBSL.

A.5 Balancing

After every step (batch operation counts as a step, but we could do rebalance after every single operation as well), the skip list would need to be rebalanced if needed. We can consider adding a limit m and a deviation δ such that a node v could only have $m - \delta \leq \text{count}(v) \leq m + \delta$, i.e. $k_{\min} = m - \delta$ and $k_{\max} = m + \delta$. Therefore, a rebalancing act could be to combine 2 side-by-side nodes that both have less than $m - \delta$ childs. Further, given nodes x, y , if $\text{count}(x) < m + \delta$ and $\text{count}(y) < m - \delta$, if $\text{count}(y) + \text{count}(x) \leq m + \delta$ we could merge y into x .

This merging could be done by a coin flip for whether y would be merged by $\text{right}(x) = \text{right}(y)$, $\text{down}(x) = y$ or we also could have $\text{right}(x) = y.\text{high}$ and remove y , where $y.\text{high}$ is the the rightmost and thus highest node in y . An important thing to note however, is that this node merging should be done from the right to left and no other way, as the nodes are cascading to the right. If 2 nodes are merged, the balancing operation cascades down in $\mathcal{O}(\log n)$ complexity as well. Note that if 2 nodes n_1, n_2 at L_i both have $\text{count}() < m - \delta$ then we would seek to balance as much as possible such that $\text{count}(n_1) \approx \text{count}(n_2)$ as close as possible. The balancing operation however is done top down, i.e. in the order of $\{L_h, L_{h-1}, L_{h-2}, \dots, L_1\}$.

Note that there will be differences in complexity when doing the balancing after a batch operation or right after every single operation. When rebalancing after a single operation, we can pinpoint the proximity effect of the operation and thus would be $\mathcal{O}(\log n)$ because it's just the surrounding nodes that are affected. This is not the case for rebalancing after a batch operation, because we cannot pinpoint the proximity affected anymore and we must then check through every node, which results in an $\mathcal{O}(2n) = \mathcal{O}(n)$ complexity to check and the same $\mathcal{O}(\log n)$ for every broken node resulting in $\mathcal{O}(n) + \mathcal{O}(u \log n)$ where u is the amount of broken nodes. Thus *rebalancing after every operation is on average better* and in the worst cases the same as rebalancing after a batch.

Definition 3 (Fan-out Invariant). An IBSL $\mathcal{S}$ satisfies the fan-out invariant if, for every

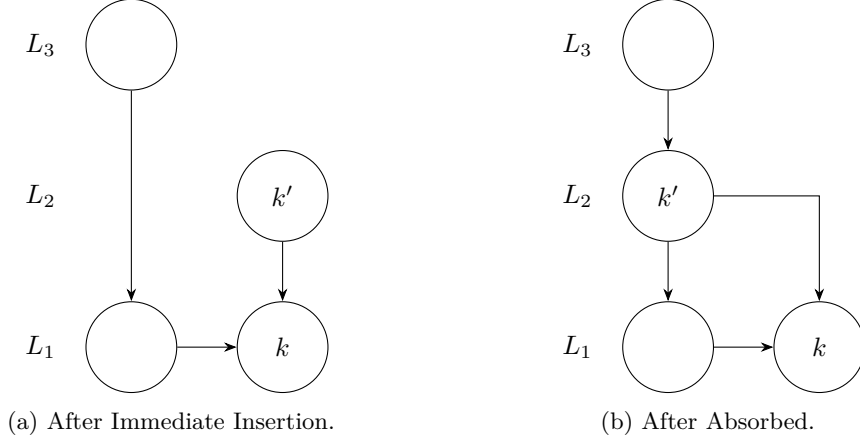


Figure 3: Insertion case 3 for the IBSL.

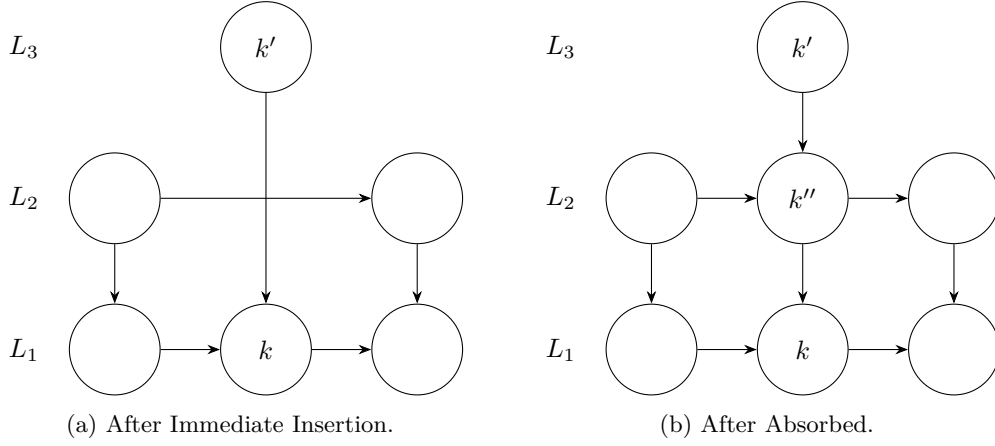


Figure 4: Insertion case 4 for the IBSL.

node $v \in \mathcal{S}$,

$$k_{\min} \leq \text{count}(v) \leq k_{\max}.$$

Lemma 6. Let $\mathcal{S}$ be an Interval Based Skip List (IBSL) over n elements. The expected complexity for a rebalance after every single operation is

$$\mathbb{E}[T(n)] = \mathcal{O}(\log n),$$

Proof. After a single operation on node v , the set of nodes affected is limited to v itself, its immediate left and right neighbors, and the corresponding nodes one level above and below v . No other node in the structure is touched. Thus a single rebalancing check after one operation only needs to examine this constant-sized neighborhood at each level, and since this check may propagate downward through at most i levels, the total cost of rebalancing after a single operation is $\mathcal{O}(\log n)$. $\square$

A.6 Proofs

Theorem 1 (Expected Height of the IBSL). Let $\mathcal{S}$ be an IBSL over n elements with promotion probability $p = 1/2$. The expected height h of $\mathcal{S}$ satisfies $h = \mathcal{O}(\log n)$.

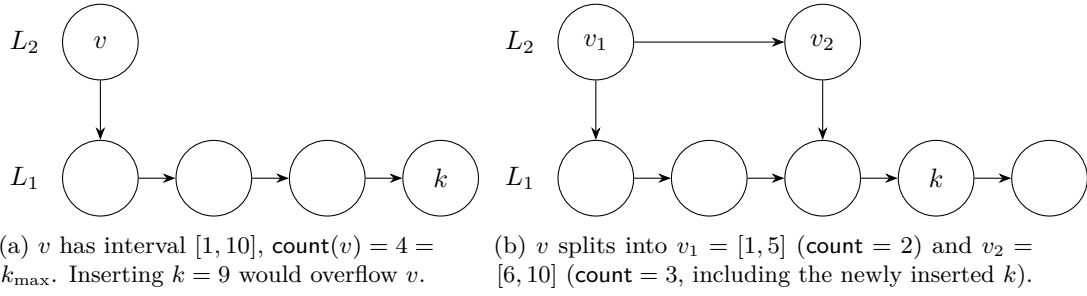


Figure 5: Overflow case: insertion forces a split when $\text{count}(v)$ would exceed $k_{\max}$.

Proof. The promotion procedure for $\mathcal{S}$ is identical to the standard skip list [Pug90], each node at level L_i is independently promoted to L_{i+1} with probability p . the amount of nodes at L_i $\mathbb{E}[|L_i|] = np^{i-1}$, which matches the level-size bound used in the original analysis. The expected height bound $\mathbb{E}[h] = O(\log_{1/p} n)$ then follows directly from the height analysis for the normal skip lists [Pug90]. The interval, the base limit m , the deviation δ bound, and shortcut-paths introduced by the IBSL (Section A) does not alter the underlying promotion mechanism of the original skip list as upon Setup the IBSL specific modifications are only done after the standard probabilistic promotions are done, and therefore does not affect the original height bound. $\square$

Theorem 2 (Expected complexity of Search of the IBSL). *Let $\mathcal{S}$ be an Interval Based Skip List (IBSL) over n elements. The expected time complexity of the Search operation on $\mathcal{S}$ is*

$$\mathbb{E}[T(n)] = \mathcal{O}(\log n),$$

where the expectation is taken over the internal randomness of the probabilistic level-promotion procedure.

Proof. Working backwards from the leaf node up to the root, unlike the standard skip list where there may be infinite left nodes at any given level L_i the IBSL has only a finite amount of left moves that is possible, which is given by the m and δ bounds which are $m + \delta$ left moves at most. Thus, complexity of Search is almost strictly bounded by the height of the IBSL. as at each level L_i , $T(n) = \mathcal{O}(m + \delta)$. Therefore for $\mathcal{O}(\log n)$ amount of levels, $\mathbb{E}[T(n)] = \mathcal{O}(m + \delta) \cdot \mathcal{O}(\log n) = \mathcal{O}(\log n)$, since $m + \delta$ is a strict upper bound, $\mathcal{O}(m + \delta) = \mathcal{O}(1)$. $\square$

Theorem 3 (Expected complexity of updates of the IBSL). *Let $\mathcal{S}$ be an Interval Based Skip List (IBSL) over n elements. The expected time complexity of Insert, and Delete operations on $\mathcal{S}$ is*

$$\mathbb{E}[T(n)] = \mathcal{O}(\log n),$$

where the expectation is taken over the internal randomness of the probabilistic level-promotion procedure.

Proof. Updates first starts with a Search to find the right node to work with before computing the update. Then for case 5 of the insertion, it will take $\mathcal{O}(\log n)$ for the cascade and $\mathcal{O}(1)$ for others. For deletion it is simply a single constant operation. Thus $\mathbb{E}[T(n)] = \mathcal{O}(\log n) + \mathcal{O}(\log n) = \mathcal{O}(\log n)$. $\square$

Theorem 4. *Let $\mathcal{S}$ be an Interval Based Skip List (IBSL) over n elements. $\mathcal{S}$ will preserve interval correctness (definition 1) throughout all cases.*

Proof. For each node $v \in \mathcal{S}$, v only has 2 states it could be in, either $right(v)$ is on the same level or is a shortcut-path. The interval correctness is derived directly from the construction. When $v' \leftarrow right(v)$ is on the same level, $max(v.interval) = min(v'.interval)$ and when $v'' \leftarrow right(v)$ is a shortcut-path, $max(v.interval) = max(v''.interval)$ which is exactly the interval correctness defined in definition 1. $\square$

Theorem 5 (Search Correctness Of The IBSL). *Let $\mathcal{S}$ be an Interval Based Skip List (IBSL) over n elements. If $\mathcal{S}$ satisfies interval Correctness (definition 1), then $\mathcal{S}$ satisfies search correctness as well for $k \in \{0, 1\}^\lambda$ (definition 2)*

Proof. When interval correctness is preserved (theorem 4), for each node $v \in \mathcal{S}$, $v.interval$ connects directly to another node leaving no gaps in any intervals. Realize that search correctness is bounded by whether $\{\exists k \in \mathcal{S} : Search(\mathcal{S}, k) = 0\}$ only as $\{\exists k \notin \mathcal{S} : Search(\mathcal{S}, k) = 1\}$ is trivial once you realize that at worst case, if $k \notin \mathcal{S}$, then there will not exist a leaf that contains k and thus in worst case Search will return 0 once it reaches L_1 .

Now remember that by construction, Search goes through each $v.interval$, $v \in \mathcal{S}$ and either goes right or down if $k \in v.interval$. If $k \in \mathcal{S}$ then it follows that there should be at least one node r in each level that has $k \in r.interval$ all the way down to L_1 where the leaf k sits. Let us proof by contradiction, assume that Search does not preserve correctness, thus this means that $\{\exists k \in \mathcal{S} : Search(\mathcal{S}, k) = 0\}$. For this to happen there must be a gap in any $v.interval$ such that Search cannot find $k \in v.interval$ and thus the path to k is cut off and Search is unable to continue the computation. However, as per theorem 4, interval correctness is preserved and thus there are no gaps in any $v.interval$, thus is a contradiction and $Search(\mathcal{S}, k)$ must always return 1 when $k \in \mathcal{S}$. $\square$

B Forest

In the forest construction like the merkle forest in zk-creds, is a forest or a set $\mathcal{F} = \{\mathcal{S}_1, \mathcal{S}_2, \dots, \mathcal{S}_n\}$. The forest construction just like the monolithic IBSL construction uses Epoch based batch operations. Furthermore, note that in (A) we detailed a bound m and δ , since this applies to all nodes, let $stor(\mathcal{S}_i)$ denote the amount of commitments (leaf nodes) are under $\mathcal{S}_i$.

B.1 Construction

Let $\mathcal{E} = \{e_0, e_1, e_2, \dots\}$ denote the (unbounded) sequence of epochs. At the start of epoch e_i , a new IBSL $\mathcal{S}_i \in \mathcal{F}$ is constructed from the commitments accumulated during the preceding epoch e_{i-1} . That is, the commitments inserted during e_3 to form $\mathcal{S}_3$ are exactly those that were received during e_2 ; $\mathcal{S}_3$ itself is built and published at the start of e_3 . All insertions, deletions, and rebalancing operations within an epoch are applied as a single batch at the epoch boundary, rather than individually as each operation arrives.

C Another Appendix Section

More content...

D AI Disclaimer

Portions of this manuscript were edited with the assistance of an AI writing tool, which was used to improve grammar, wording, and formatting consistency. All technical content—including definitions, theorems, and proofs—was produced and verified by the authors, who take full responsibility for the correctness and originality of the work.